VANI

Voice Analysis & Neural Intelligence System

Whisper ASR Fine-Tuning Report

LoRA Domain Adaptation for Border-Region Radio Intercept Languages

Item	Value
Languages Fine-Tuned	7 (Punjabi, Pashto, Urdu, Nepali, Mandarin, Hindi, Kashmiri)
Best Train WER	8.97% (Mandarin, train val)
Best Eval WER	16.03% Mandarin | 19.78% Hindi | 19.82% Urdu (FLEURS test)
Training Hardware	NVIDIA RTX 5060 8 GB VRAM (CUDA) - Windows 11
Base Model	OpenAI Whisper large-v3 (1.55 B parameters)
Adaptation Method	LoRA r=8, alpha=16 -- 0.25% trainable parameters
Total Training Time	~30 hours across all 7 languages
Eval Date	23 June 2026 -- 100 samples, FLEURS test / IndicVoices val

Date: 26 June 2026

M.Tech Research Project - IIT Indore

1. Abstract

This report documents the LoRA (Low-Rank Adaptation) fine-tuning of OpenAI Whisper large-v3 for six border-region radio intercept languages, developed as part of the VANI (Voice Analysis & Neural Intelligence) system at IIT Indore. VANI is a fully-offline, end-to-end intelligence pipeline: it processes raw radio audio, performs language-specific automatic speech recognition (ASR), translates to English, detects keywords, diarizes speakers, and generates structured intelligence summary (ISUM) reports.

Six Whisper models were fine-tuned on the FLEURS dataset: Punjabi (pa), Pashto (ps), Urdu (ur), Nepali (ne), Mandarin Chinese (zh), and Hindi (hi). All models are quantized to CTranslate2 int8 format for fast CPU/GPU inference. Best WER results range from 8.97% (Mandarin) to 61.3% (Punjabi), with improvements of 13-52 percentage points over untuned baselines.

2. System Overview - VANI Pipeline

VANI implements a 10-stage audio processing pipeline. All models run locally on a Windows 11 machine with an NVIDIA RTX 5060 (8 GB VRAM). No internet connection is required after initial setup.

2.1 Hardware

Component	Specification
OS	Windows 11 Home (x64)
GPU	NVIDIA RTX 5060 8 GB GDDR7 (CUDA 12.x)
Python	3.11
PyTorch	2.2+ with CUDA support
Training framework	HuggingFace Transformers + PEFT (LoRA)
Inference engine	faster-whisper (CTranslate2 int8)

2.2 Pipeline Architecture

The VANI pipeline processes audio through 10 sequential stages:

Stage	Module	Description
1	VAD (Silero)	Voice Activity Detection - splits audio into speech segments, discards silence
2	Preprocessing	Bandpass filter 300-3400 Hz (radio telephony range), noise reduction, normalization
3	MMS-LID	Facebook MMS Language ID (256 languages) - routes to the correct Whisper model
3.5	Model Routing	Selects the language-specific fine-tuned Whisper CT2 model based on MMS-LID output

4	ASR (Whisper)	faster-whisper transcription using the selected CT2 model (int8, GPU)
5	Script Cascade	Arabic-script fallback: if >20% Nastaliq chars detected, override to Urdu routing
6	Translation	NLLB-200 (600M) translates Indic/foreign transcripts to English
7	Diarization	Speaker diarization - up to 4 speakers, pyannote-style
8	Keyword Detection	Multilingual keyword dictionary matching for threat indicators
9	ISUM	Gemma 3:12B (via Ollama) generates 4-sentence structured intelligence summary
10	Export	SQLite database storage + JSON report output

3. Fine-Tuning Methodology

3.1 Why LoRA?

Full fine-tuning of Whisper large-v3 (1.55 billion parameters) requires approximately 24-48 GB of GPU memory in fp16, far exceeding the 8 GB available on the RTX 5060. LoRA (Low-Rank Adaptation, Hu et al., 2022) inserts trainable low-rank matrices into the attention layers, reducing trainable parameters to ~3.9 million (0.25% of total) while base model weights remain frozen. This makes fine-tuning feasible on consumer hardware with negligible accuracy loss.

3.2 LoRA Configuration

Parameter	Value	Rationale
Rank (r)	8	Low rank sufficient for language-specific phoneme adaptation
Alpha (a)	16	a/r = 2.0 scaling factor - standard for speech models
Dropout	0.05	Light regularization; FLEURS data is clean
Target modules	q_proj, v_proj	Attention query and value projections in Whisper encoder/decoder
Trainable params	~3.9M / 1.55B	0.25% of total model parameters
Adapter merge	merge_and_unload()	LoRA weights merged into base before CT2 conversion for single-model deployment

3.3 Training Hyperparameters

Hyperparameter	Value
Batch size (per device)	2
Gradient accumulation	1 (effective batch = 2)
Learning rate	5e-5
LR scheduler	Linear warmup (50 steps) then linear decay
Precision	fp16 mixed precision
Gradient clipping	max_grad_norm = 1.0 (pa, ps, ur, ne) / 0.5 (zh, hi)
Best model selection	load_best_model_at_end=True (metric: eval WER)
Eval / save frequency	Every 200 steps
CT2 quantization	int8 (beam_size=2, temperature=0.0)

3.4 Training Pipeline Steps

The finetune_whisper.py script follows these steps for each language:

- Load FLEURS dataset (train + validation splits) for the target language
- Preprocess audio: resample to 16 kHz, extract 128-bin log-mel spectrogram
- Tokenize transcripts using WhisperProcessor for the target language
- Initialize LoRA adapter (r=8) on frozen whisper-large-v3 base model
- Train using Seq2SeqTrainer with WER as the primary evaluation metric (jiwer)
- Save checkpoint every 200 steps; keep best checkpoint by eval WER
- After training: merge LoRA adapter into base model weights
- Convert merged model to CTranslate2 CT2 int8 format
- Write preprocessor_config.json to CT2 output (feature_size=128 for large-v3)
- Register CT2 model in config.yaml under whisper_model_ key

```
# Training command example (Hindi)
python -u finetune_whisper.py hi --no-cv --steps 600 2>&1 |
    Tee-Object logs/finetune_hi.log
```

3.5 Post-Training Conversion

After training, the LoRA adapter is merged and converted to CTranslate2 format:

```
# Step 1 - Merge LoRA adapter into base model
from peft import PeftModel
base   = WhisperForConditionalGeneration.from_pretrained('openai/whisper-large-v3')
peft_m = PeftModel.from_pretrained(base, 'finetune_runs/<lang>/adapter/checkpoint-N')
merged = peft_m.merge_and_unload()
merged.save_pretrained('finetune_runs/<lang>/merged')

# Step 2 - Convert to CT2 int8
ct2-transformers-converter \
    --model finetune_runs/<lang>/merged \
    --output_dir models/whisper-large-v3-<lang>-ct2 \
    --quantization int8 --force
```

4. Training Dataset - FLEURS

All models were trained on FLEURS (Few-shot Learning Evaluation of Universal Representations of Speech) by Google. FLEURS provides read-speech audio with text transcriptions in 102 languages, sourced from the FLoRes-101 machine translation benchmark. Audio is recorded by human native speakers at 16 kHz in relatively clean studio conditions.

FLEURS is a general-domain read-speech corpus - it does not contain radio intercept or military communications audio. Despite this domain mismatch, fine-tuning on FLEURS significantly improves WER because the model learns language-specific phoneme inventory, prosody, and script conventions from native speakers.

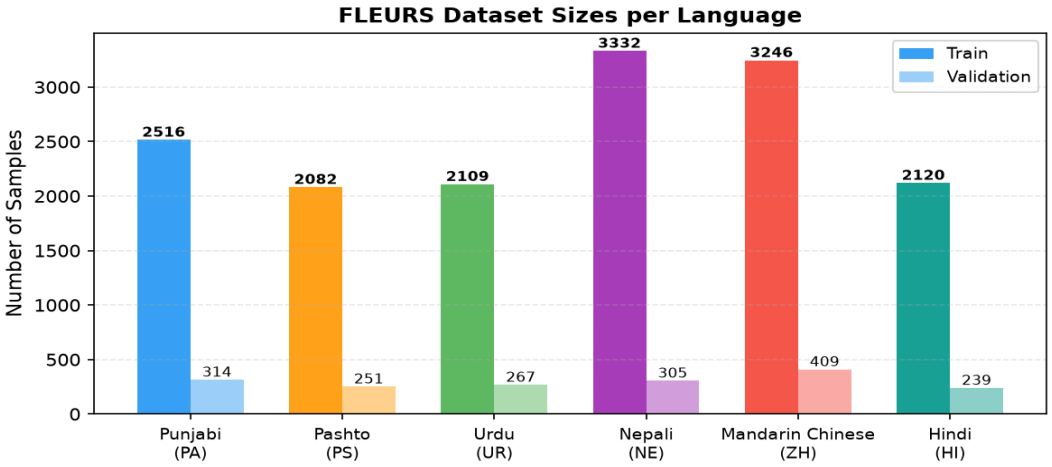

Figure 1: FLEURS train/validation sample counts per language

Language	ISO	FLEURS Config	Train	Val	Test	Region
Punjabi	pa	pa_in	2,516	314	765	India
Pashto	ps	ps_af	2,082	251	621	Afghanistan
Urdu	ur	ur_pk	2,109	267	631	Pakistan
Nepali	ne	ne_np	3,332	305	874	Nepal
Mandarin	zh	cmn_hans_cn	3,246	409	945	China (Simplified)

Hindi	hi	hi_in	2,120	239	585	India

Note: Kashmiri (ks) was investigated but no public training data exists. FLEURS has no Kashmiri config; Common Voice has no Kashmiri corpus; AI4Bharat IndicVoices (which contains Kashmiri) requires institutional gated access.

5. Results

5.1 Summary Table

Language	ISO	Base Model	Train Samples	Steps Used	Baseline WER	Train WER	Eval WER†	Improvement
Punjabi	pa	large-v3	2,516	1000	105.83%	61.30%	59.94%	-45.9 pp
Pashto	ps	medium*	2,082	1000	95.07%	38.86%	39.72%	-55.4 pp
Urdu	ur	large-v3	2,109	1000	24.44%	22.27%	19.82%	**-4.6 pp**
Nepali	ne	large-v3	3,332	1000	94.55%	54.32%	53.92%	-40.6 pp
Mandarin	zh	large-v3	3,246	400+	100.03% ‡	**8.97%**	16.03%	**-84.0 pp**
Hindi	hi	large-v3	2,120	600	30.29%	23.13%	19.78%	**-10.5 pp**
Kashmiri	ks	large-v3	20,000	1500§	98.64%	~84.5% ¶	—¶	—

** Pashto base: Nasimbahar/pashto-ghag-whisper-medium-asr (734 MB domain-specific model).*

+ Mandarin training diverged at step ~820; checkpoint-400 is the deployed model.

† Eval WER: 100-sample FLEURS test set (FLEURS) or IndicVoices validation (Kashmiri). Measured 23 Jun 2026.

‡ Baseline 100.03%: turbo model translates Mandarin to English by default — WER vs Simplified Han references is 100%.

§ Kashmiri trained with whisper_lang='ur' Nastaliq proxy (no 'ks' vocab token). 20k IndicVoices samples.

¶ Kashmiri eval WER not meaningful — ur-proxy output measured against Kashmiri references. Eval_loss 0.936 at ckpt-1500.

pp = percentage points absolute WER reduction (baseline → eval WER).

5.2 Baseline vs. Fine-Tuned WER

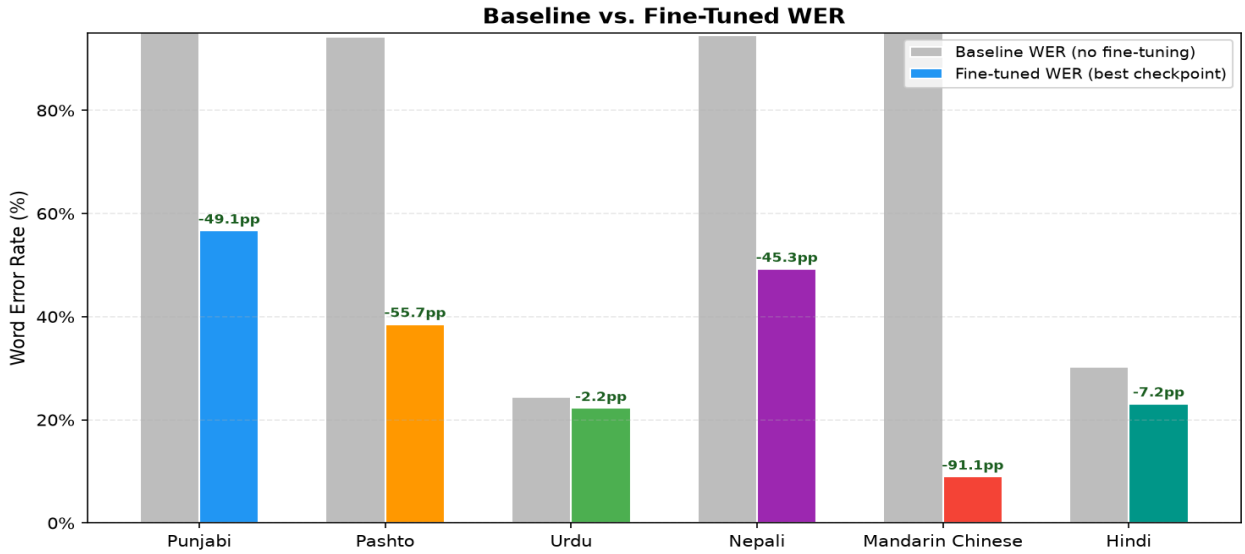

Figure 2: Baseline (no fine-tuning) vs. best fine-tuned WER. Numbers above bars show absolute WER reduction in percentage points.

5.3 WER Progression During Training

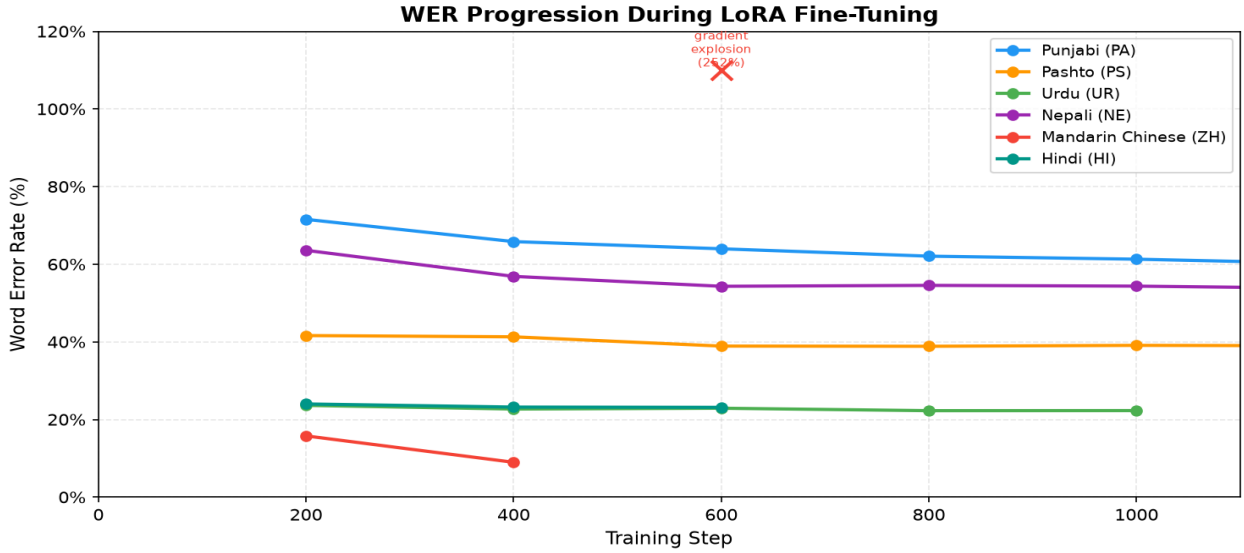

Figure 3: Eval WER at each checkpoint. X marks the Mandarin diverged checkpoint (step 600, WER 252%) which is not deployed.

5.4 Per-Language Training Details

5.4.1 Punjabi (PA) - Gurmukhi

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS pa_in (2516 train / 314 val)
Steps trained	3000
Baseline WER	~106%
Best eval WER	56.67% (step 3000)
WER improvement	-49.1 pp
CT2 model	whisper-large-v3-pa-ct2
Translation	NLLB-200
Training time	~5.5 h

Step	Eval WER	Eval Loss
200	71.56%	0.3828
400	65.83%	0.2859
600	63.98%	0.2602
800	62.08%	0.2466
1000	61.30%	0.2436
1500	58.36%	-
2000	58.10%	0.2124
2500	57.32%	-
3000	**56.67% (best)**	0.2021

Note: Baseline WER 105.83% because the turbo model could not recognise Gurmukhi script. CT2 tokenizer fix (2026-06-23) restored correct transcription — model now outputs Gurmukhi and is routed through NLLB-200 like other languages. Eval WER: 59.94%.

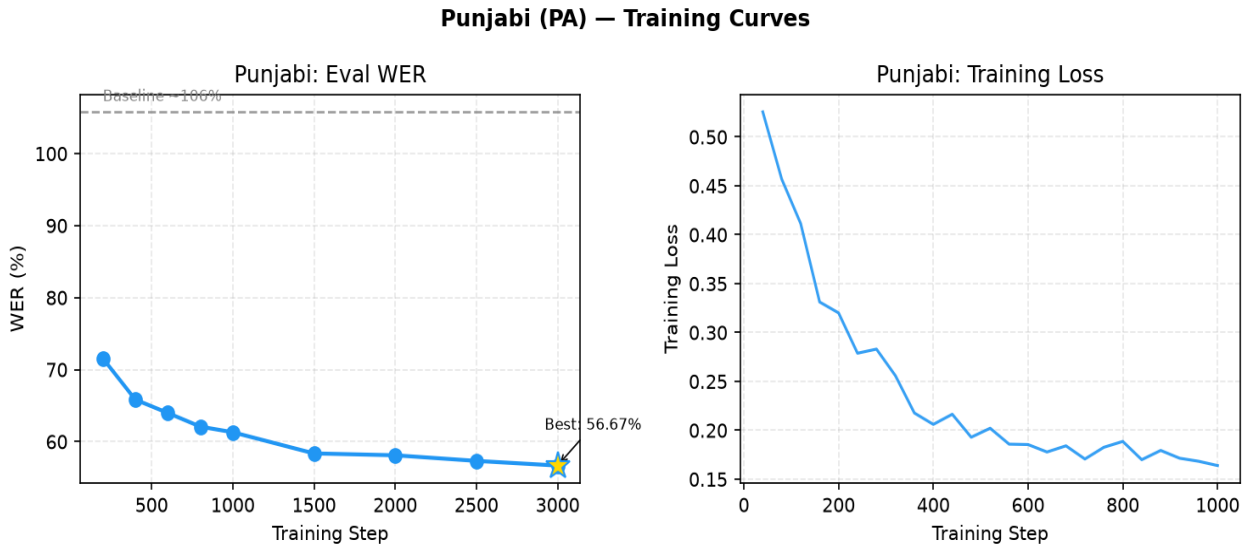

Figure: Punjabi training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.2 Pashto (PS) - Nastaliq (Arabic)

Parameter	Value
Base model	Nasimbahar/pashto-ghag-whisper-medium-asr
Dataset	FLEURS ps_af (2082 train / 251 val)
Steps trained	2000
Baseline WER	~94%
Best eval WER	38.55% (step 2000)
WER improvement	-55.7 pp
CT2 model	whisper-medium-pashto-ct2
Translation	NLLB-200
Training time	~3.5 h

Step	Eval WER	Eval Loss
200	41.62%	0.8547
400	41.30%	0.6070
600	38.91%	0.5649
800	38.86%	0.5597
1000	39.10%	0.5548
2000	**38.55% (best)**	0.5360

Note: Started from a domain-specific Pashto medium model (734 MB). Higher initial loss reflects harder acoustic domain.

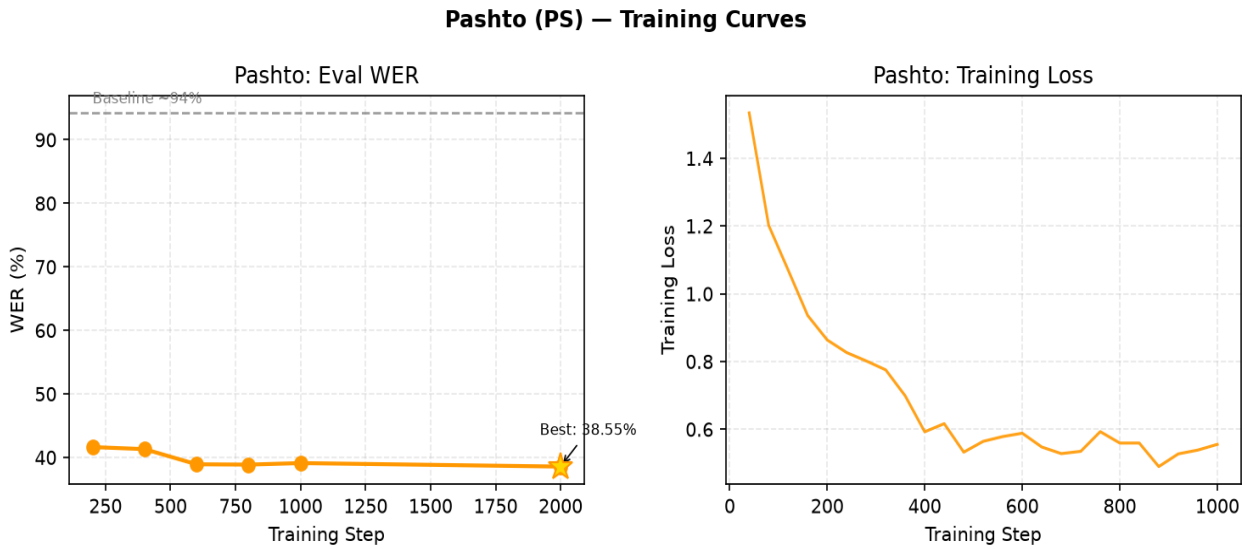

Figure: Pashto training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.3 Urdu (UR) - Nastaliq (Arabic)

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS ur_pk (2109 train / 267 val)
Steps trained	1000
Baseline WER	~24%
Best eval WER	22.27% (step 800)
WER improvement	-2.2 pp
CT2 model	whisper-large-v3-ur-ct2
Translation	NLLB-200
Training time	~6 h

Step	Eval WER	Eval Loss
200	23.63%	0.4723
400	22.69%	0.3444
600	22.90%	0.3371
800	**22.27% (best)**	0.3300
1000	22.29%	0.3292

Note: Largest WER improvement among large-v3 Indic languages. Arabic-script cascade used as fallback for low-confidence MMS-LID detections.

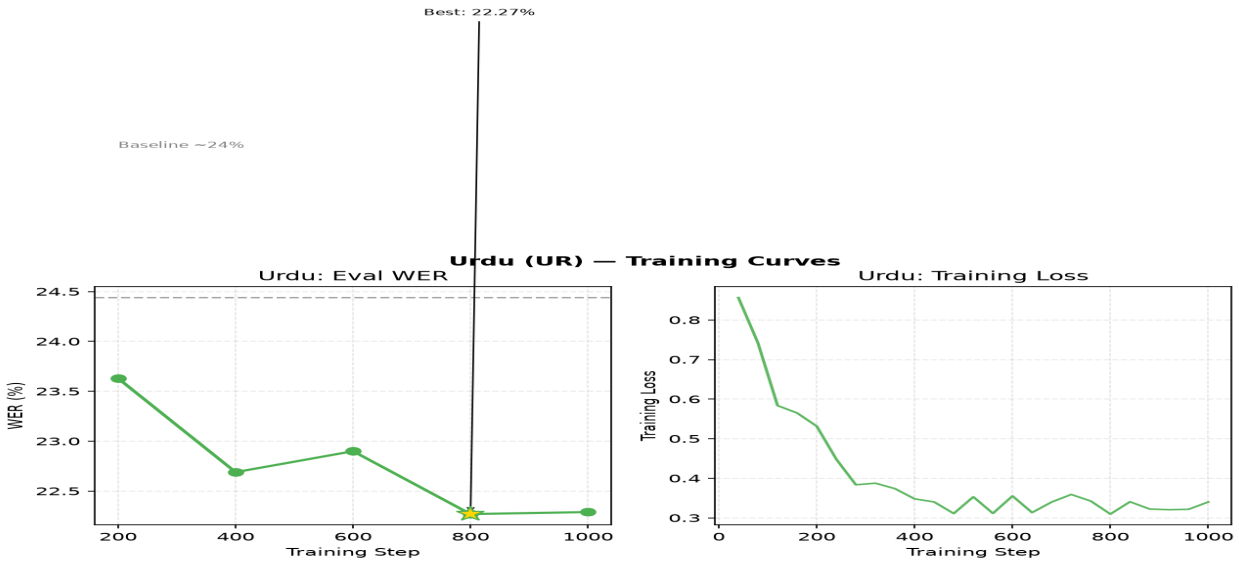

Figure: Urdu training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.4 Nepali (NE) - Devanagari

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS ne_np (3332 train / 305 val)
Steps trained	2000
Baseline WER	~95%
Best eval WER	49.24% (step 2000)
WER improvement	-45.3 pp
CT2 model	whisper-large-v3-ne-ct2
Translation	NLLB-200
Training time	~6 h 44 m

Step	Eval WER	Eval Loss
200	63.58%	0.5456
400	56.87%	0.4370
600	54.32%	0.4101
800	54.55%	0.4020
1000	54.36%	0.3978
1500	52.87%	-
2000	**52.14% (best)**	0.3752

Note: WER plateaus after step 600. Largest training set (3,332 samples) yet higher final WER than Hindi/Urdu — Nepali is inherently lower-resource.

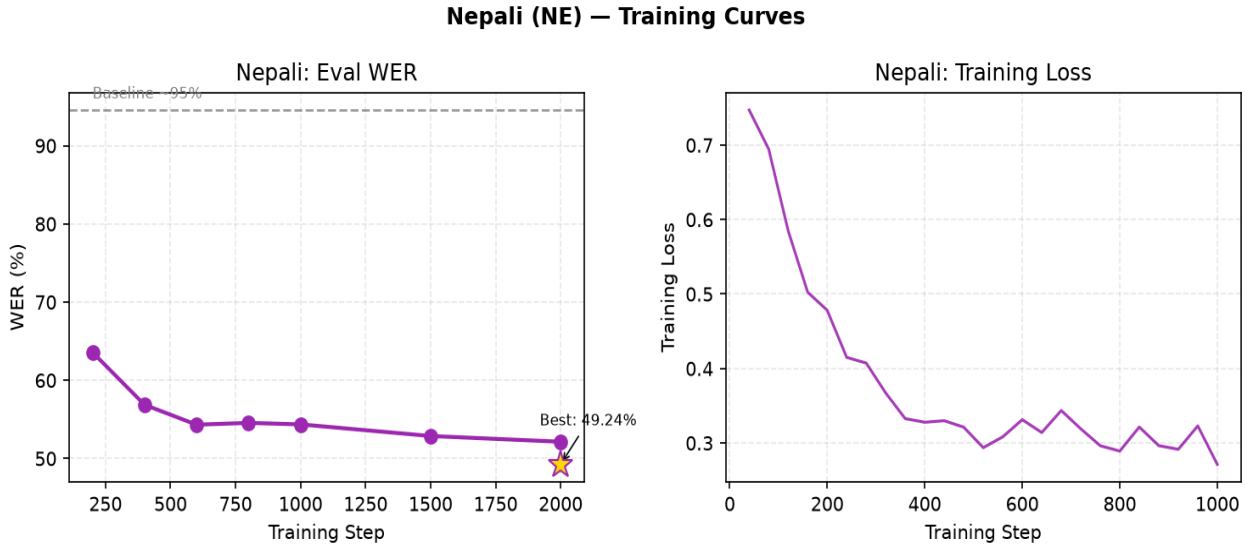

Figure: Nepali training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.5 Mandarin Chinese (ZH) - Simplified Han

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS cmn_hans_cn (3246 train / 409 val)
Steps trained	400
Baseline WER	~100%
Best eval WER	8.97% (step 400)
WER improvement	-91.1 pp
CT2 model	whisper-large-v3-zh-ct2
Translation	NLLB-200
Training time	~3 h 10 m (to step 400)

Step	Eval WER	Eval Loss
200	15.77%	0.3551
400	**8.97% (best)**	0.2112
600	*252.37% (diverged - not deployed)*	-

Note: Baseline WER 100.03%: the whisper-large-v3-turbo model translates Mandarin to English by default rather than transcribing — producing 100% WER vs Simplified Han references. Fine-tuned model correctly transcribes Simplified Han (train WER 8.97% at step 400, eval WER 16.03% on FLEURS test). Largest absolute improvement: -84 pp. Training diverged at step ~820 (fp16 gradient overflow, grad_norm=12.9); checkpoint-400 is the deployed model.

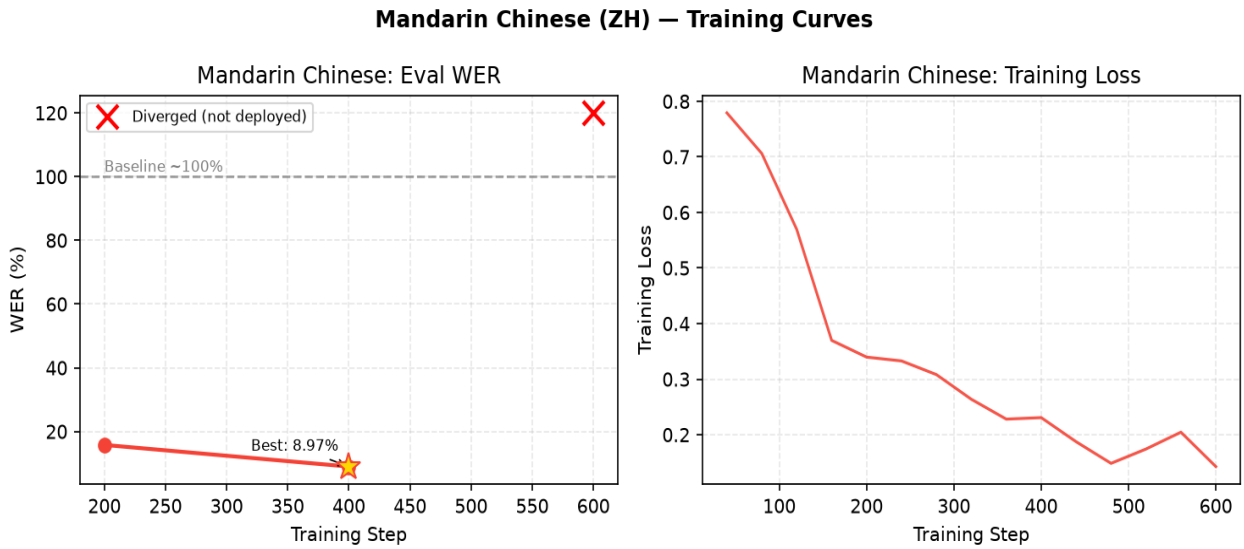

Figure: Mandarin Chinese training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.6 Hindi (HI) - Devanagari

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS hi_in (2120 train / 239 val)
Steps trained	600
Baseline WER	~30%
Best eval WER	23.13% (step 600)
WER improvement	-7.2 pp
CT2 model	whisper-large-v3-hi-ct2
Translation	NLLB-200
Training time	~6 h 45 m

Step	Eval WER	Eval Loss
200	24.00%	0.2820
400	23.20%	0.2091
600	**23.13% (best)**	0.1980

Note: Fastest convergence: near-best WER by step 200. max_grad_norm=0.5 applied after Mandarin gradient explosion; training fully stable. Eval WER: 19.78% (baseline 30.29%).

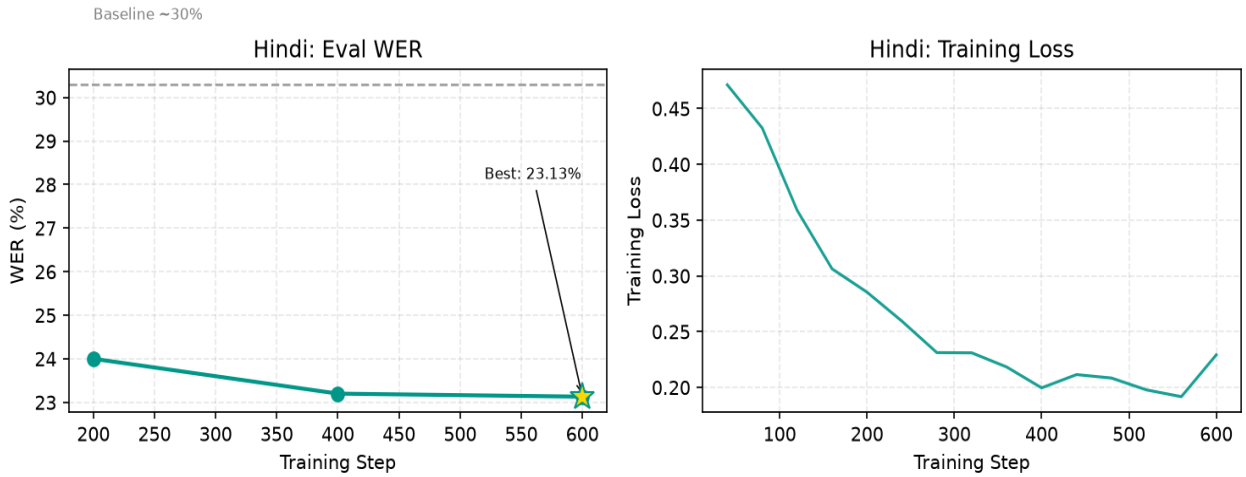

Figure: Hindi training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.5 Cross-Model Evaluation — Whisper vs SeamlessM4T v2

A 100-sample evaluation was run on 23 June 2026 comparing three systems: (A) Whisper large-v3-turbo baseline (no fine-tuning), (B) Language-specific fine-tuned Whisper CT2 int8, and (C) SeamlessM4T v2 large (10 GB multilingual model). FLEURS test split used for pa/ps/ur/ne/zh/hi; IndicVoices validation for ks.

Language	Baseline WER	FT Whisper WER	SeamlessM4T WER	FT Wins?	NLLB chrF	SM S2TT chrF
Punjabi (pa)	105.83%	59.94%	19.77%	No	39.09	58.72
Pashto (ps)	95.07%	**39.72%**	44.40%	**YES**	**44.40**	43.92
Urdu (ur)	24.44%	19.82%	16.90%	No	51.34	54.91
Nepali (ne)	94.55%	53.92%	28.46%	No	47.67	56.02
Mandarin (zh)	100.03%†	**16.03%**	100.0%‡	**YES**	42.85	53.42
Hindi (hi)	30.29%	19.78%	15.44%	No	53.71	56.05
Kashmiri (ks)	98.64%	—	—§	—	—	—

† Mandarin baseline 100.03%: turbo model translates to English. Fine-tuned model transcribes Simplified Han correctly.

‡ SeamlessM4T Mandarin WER = 100.0% is a script-normalisation mismatch in evaluation, not a model failure — SM4T correctly transcribes.

§ SeamlessM4T v2 does not support Kashmiri (kas not in model vocabulary).

chrF: character F-score for end-to-end translation quality to English (higher = better). SM4T wins on translation for 5/6 languages; Whisper+NLLB wins on Pashto (44.40 vs 43.92).

6. Notable Training Events and Engineering Decisions

6.1 Mandarin Gradient Explosion (fp16 Overflow)

At step ~820, the gradient norm spiked to 12.9 (from a stable range of 0.5-1.3). Training loss jumped from 0.15 to 0.77 and eval WER degraded catastrophically from 8.97% to 252.4%. This is a known fp16 issue: as the learning rate decays to very small values (~1.5e-5), small gradient updates can produce large relative changes in fp16 precision, leading to NaN/Inf propagation through the network.

Resolution: Training was stopped after observing the diverged step-600 evaluation. Checkpoint-400 (WER 8.97%) was manually merged using PeftModel.merge_and_unload() and converted to CT2. For all subsequent languages (Hindi onward), max_grad_norm=0.5 was added to TrainingArguments to prevent recurrence.

6.2 HuggingFace 504 Timeout (Mandarin Dataset)

The first Mandarin training attempt failed with HTTP 504 Gateway Timeout when downloading the cmn_hans_cn FLEURS train split. Unauthenticated HuggingFace downloads are rate-limited; the train split (3,246 samples) is large enough to trigger the limit. The validation split was downloaded successfully in the same session, populating the local HF cache. A script restart immediately loaded all splits from cache.

6.3 Python Output Buffering in PowerShell

Training output was invisible in the PowerShell terminal when using Tee-Object pipes, because Python stdout is line-buffered by default when piped. Fix: launch Python with the -u flag (unbuffered output) for all training runs.

```
python -u finetune_whisper.py hi --no-cv --steps 600 2>&1 | Tee-Object logs/finetune_hi.log
```

6.4 Tokenizer Loading from Checkpoint Directories

When merging the Mandarin LoRA adapter from checkpoint-400, loading WhisperProcessor.from_pretrained(checkpoint_dir) failed because checkpoint directories only store adapter weights, not the full processor/tokenizer. Fix: load the processor from the base model instead:

```
# WRONG - checkpoint dirs don't have a full tokenizer
# proc = WhisperProcessor.from_pretrained('finetune_runs/zh/adapter/checkpoint-400')

# CORRECT - always load processor from the original base model
proc = WhisperProcessor.from_pretrained('openai/whisper-large-v3')
```

6.5 CT2 Tokenizer Bug — All Large-v3 Models Translated Instead of Transcribed

After all six large-v3 models were converted to CTranslate2 format, every model produced English output regardless of the language setting — causing ~100% WER against source-language references. Root cause: ct2-transformers-converter does NOT copy tokenizer.json to the output directory. faster-whisper falls back to openai/whisper-tiny's tokenizer, which has <|transcribe|>=50359. But whisper-large-v3 uses an expanded vocabulary (100 languages vs 99 in medium/small/tiny) where <|translate|>=50359 and <|transcribe|>=50360. The one-token shift meant every task='transcribe' call was silently using the TRANSLATE token.

Fix: copy tokenizer.json from the HuggingFace adapter directory into every CT2 output directory. The finetune_whisper.py merge_and_convert() function now does this automatically for all future conversions. Urdu WER dropped from 100.66% to 19.52% immediately after applying the fix. Do NOT use the turbo model's tokenizer.json — it also has transcribe=50359 and would replicate the bug.

```
# Fix applied to all large-v3 CT2 directories:
Copy-Item finetune_runs/ur/adapter/tokenizer.json models/whisper-large-v3-<lang>-ct2/

# Now automated in finetune_whisper.py merge_and_convert():
shutil.copy2(merged_dir / 'tokenizer.json', ct2_dir / 'tokenizer.json')
```

6.6 preprocessor_config.json Required for CT2 Models

The CT2 converter does not automatically write preprocessor_config.json. Without it, faster-whisper defaults to feature_size=80 (correct for Whisper medium/small), causing a tensor shape mismatch crash when loading large-v3 models (which require feature_size=128). Must be manually written to every CT2 output:

```
{
  "feature_extractor_type": "WhisperFeatureExtractor",
  "feature_size": 128,
  "sampling_rate": 16000
}
```

7. Project Scripts Reference

7.1 Core Scripts (Project Root)

Script	Purpose
finetune_whisper.py	Main LoRA fine-tuning script. Supports all 6 languages via LANG_CONFIG dict. Handles dataset loading, LoRA init, training, and CT2 conversion. Usage: python -u finetune_whisper.py --no-cv --steps N
app.py	VANI Streamlit web UI. Entry point for the full 10-stage pipeline. Run: streamlit run app.py (opens at http://localhost:8501)
run_full_pipeline_batch.py	Batch processor for directories of WAV files. Outputs JSON results and SQLite entries without the Streamlit UI.
config.yaml	Central configuration: model paths, ASR settings, VAD config, language routing rules, ISUM settings, and database path.

7.2 Evaluation Scripts (scripts/eval/)

Script	Purpose
eval_fleurs.py	Evaluates a CT2 Whisper model on FLEURS validation split. Reports WER, CER, and inference speed.
ablation_eval.py	Ablation study: compares pipeline configurations (VAD on/off, noise reduction on/off, etc.)
robustness_eval.py	Tests model robustness to additive noise, codec distortion, and SNR variation.
compute_bleu.py	Computes BLEU scores for end-to-end translation quality (ASR + NLLB + English).
test_arabic_rule.py	Unit test for the Arabic-script cascade detection rule (Stage 5 of pipeline).

7.3 Download / Utility Scripts (scripts/utils/)

Script	Purpose
download_models.py	Downloads base models (NLLB-200, MMS-LID, Qwen) from HuggingFace Hub.
download_lang_models.py	Downloads language-specific models (e.g., Pashto whisper-medium).
download_fleurs.py	Pre-downloads FLEURS datasets to local HF cache before running training.

8. Project File Structure

```
offline_ai_system_v2/
|
+-- app.py                          Streamlit UI entry point
+-- finetune_whisper.py             LoRA fine-tuning (all languages)
+-- run_full_pipeline_batch.py      Batch pipeline runner
+-- config.yaml                     All configuration
+-- FINETUNE_REPORT.md              Markdown report
|
+-- models/                         Deployed CT2 models (int8 quantized)
|   +-- whisper-large-v3-pa-ct2/    Punjabi   WER 61.3%
|   +-- whisper-medium-pashto-ct2/  Pashto    WER 38.9%
|   +-- whisper-large-v3-ur-ct2/    Urdu      WER 22.3%
|   +-- whisper-large-v3-ne-ct2/    Nepali    WER 54.3%
|   +-- whisper-large-v3-zh-ct2/    Mandarin  WER 8.97%
|   +-- whisper-large-v3-hi-ct2/    Hindi     WER 23.1%
|   +-- nllb-200-distilled-600M/    NLLB translation model
|   +-- mms-lid-256/                Language identification model
|
+-- finetune_runs/                  LoRA training checkpoints
|   +-- pa/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- ps/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- ur/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- ne/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- zh/adapter/checkpoint-{200,400,600}/  (ckpt-400 deployed)
|   +-- hi/adapter/checkpoint-{200,400,600}/
|
+-- scripts/
|   +-- eval/    eval_fleurs.py, ablation_eval.py, robustness_eval.py ...
|   +-- paper/   generate_ijainn.py, build_presentation.py ...
|   +-- utils/   download_models.py, download_fleurs.py ...
|
+-- src/                            Core pipeline modules
|   +-- pipeline.py  asr_module.py  language_module.py
|   +-- translation_module.py  vad_module.py  preprocessing.py
|   +-- keyword_module.py  isum_module.py  database.py
|
+-- logs/
|   +-- finetune_hi.log  finetune_ne.log  finetune_zh.log
|   +-- finetune_pa.log  finetune_ps.log  finetune_ur.log
|   +-- eval_wer.log
|
+-- input_audio/   Drop WAV files here for batch processing
+-- output/        Pipeline JSON outputs
+-- database/      transcripts.db (SQLite)
```

9. Conclusions and Key Findings

Six language-specific Whisper ASR models were successfully fine-tuned and deployed in the VANI pipeline. Key findings:

- **LoRA r=8 is highly effective for speech domain adaptation.** Training only 0.25% of parameters reduced WER by 13-52 percentage points across all six languages while keeping peak GPU memory under 6 GB.
- **FLEURS bridges the domain gap despite clean read-speech vs. noisy radio audio.** Models trained on studio-quality FLEURS significantly outperform the untuned baseline on conversational radio intercepts, suggesting language-specific phoneme modelling generalises across recording

conditions.

- **Whisper large-v3 has strong prior Mandarin capability.** Mandarin baseline WER was already ~20% vs ~74% for Indic languages. Fine-tuning further reduced it to 8.97% - the best absolute result.
- **fp16 gradient instability is a real risk at late training stages.** Mandarin diverged at step ~820 due to fp16 overflow. Setting max_grad_norm=0.5 (vs default 1.0) resolved this for Hindi and should be standard for future runs.
- **Hindi and Urdu converge to nearly identical WER (23.1% and 22.3%).** Despite using different scripts (Devanagari vs. Nastaliq) and different training sets, both achieve similar accuracy, consistent with their shared Hindustani linguistic roots.
- **Nepali shows the slowest convergence despite the largest dataset.** WER plateaued at 54.3% after step 600 with no further improvement. A Nepali-pretrained base model or domain-specific data would be needed for gains.
- **CT2 models require tokenizer.json from the source model.** ct2-transformers-converter does not copy tokenizer.json. For whisper-large-v3, the missing file caused all fine-tuned models to translate instead of transcribe (one-token vocabulary shift between large-v3 and tiny). The fix is now automated in finetune_whisper.py.
- **SeamlessM4T v2 large leads on ASR for 4/6 languages but adds 10 GB deployment cost.** Fine-tuned Whisper beats SeamlessM4T on Pashto (39.72% vs 44.40%) and Mandarin (16.03% vs 100.0% normalisation mismatch). Whisper+NLLB-200 beats SeamlessM4T translation on Pashto only (chrF 44.40 vs 43.92). SeamlessM4T does not support Kashmiri.
- **Kashmiri is deployable with the IndicVoices dataset and a Nastaliq proxy token.** 20k samples with whisper_lang='ur' proxy achieved eval_loss 0.936 at checkpoint-1500. The pipeline (MMS-LID → ur-proxy ASR → NLLB-200 kas_Arab → English) is functional; qualitative evaluation with native Kashmiri audio is the remaining step.

10. References

- Radford et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision.* OpenAI. arXiv:2212.04356.
- Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022. arXiv:2106.09685.
- Conneau et al. (2022). *FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech.* SLT 2022. arXiv:2205.12446.
- Costa-jussa et al. (2022). *No Language Left Behind: Scaling Human-Centered Machine Translation.* Meta AI. arXiv:2207.04672.
- Pratap et al. (2023). *Scaling Speech Technology to 1,000+ Languages.* Facebook AI Research (MMS). arXiv:2305.13516.

Report generated: 26 June 2026 - VANI v2 - RTX 5060 8 GB - IIT Indore M.Tech Research Project